Ann prac1

```
import numpy as np

# ----------------------------
# Activation Functions
# ----------------------------
def sigmoid(x):
   return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
   return x * (1 - x)

# ----------------------------
# Input Data
# ----------------------------
X_base = np.array([
   [0,0],
   [0,1],
   [1,0],
   [1,1]
])

# ----------------------------
# Logic Gates Outputs
# ----------------------------
gates = {
   "AND": np.array([[0],[0],[0],[1]]),
   "OR": np.array([[0],[1],[1],[1]]),
   "XOR": np.array([[0],[1],[1],[0]]),
   "NAND":np.array([[1],[1],[1],[0]]),
   "NOR": np.array([[1],[0],[0],[0]])
```

```python
}

learning_rate = 0.1
epochs = 10000

# -----------------------------
# Training for Each Gate
# -----------------------------
np.random.seed(0)

for gate, y in gates.items():

    # Initialize weights
    W1 = np.random.rand(2, 4)
    b1 = np.zeros((1,4))
    W2 = np.random.rand(4, 1)
    b2 = np.zeros((1,1))

    # Training loop
    for _ in range(epochs):
        hidden = sigmoid(np.dot(X_base, W1) + b1)
        output = sigmoid(np.dot(hidden, W2) + b2)

        # Backpropagation
        error = y - output
        d_output = error * sigmoid_derivative(output)

        error_hidden = d_output.dot(W2.T)
        d_hidden = error_hidden * sigmoid_derivative(hidden)

        # Update weights
```

```
    W2 += hidden.T.dot(d_output) * learning_rate
    W1 += X_base.T.dot(d_hidden) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

# Testing
hidden = sigmoid(np.dot(X_base, W1) + b1)
test_output = sigmoid(np.dot(hidden, W2) + b2)

# Output
print("\nLogic Gate:", gate)
print("A B Output")

for i in range(4):
    print(X_base[i][0], X_base[i][1], round(test_output[i][0]))
```